Dark Edges

The Gemma 4 code graphs miss method calls, not functions,
and filling them in does not fix localisation

Soumi Sharma
Indian Institute of Science, Bengaluru

Gemma 4 Developer Agent: Paper Track, November 2026

Abstract

The Gemma 4 Developer Agent competition gives every agent a precomputed code graph: a map of which function calls which. I ran the failing tests of 103 tasks under a lightweight tracer to see which real calls the map misses, and whether that matters for finding the bug. Between 47% (fastapi) and 81% (rich) of runtime call pairs have no edge in the graph. Calls *into methods* are almost never in the graph (87–100% missing), while calls into plain functions mostly are (12–17% missing); repositories differ only because some call methods more. The function that the real fix edits is executed by the failing tests in 81–83% of tasks, but lies within three graph hops of the failure in only 24–26%. A short static pass over the node text recovers 41% of the missing edges but does not improve graph-based localisation. Runtime evidence does help, but mostly through the failing test: tests that already exist run the edited function in only 52% of tasks, and fusing them with text search adds only 6 points of recall@20 (not significant). In a pre-registered test, a tracing tool did not help a 4B Gemma agent, which called it in about a quarter of runs. The runtime evidence that matters is a failing reproduction, and getting a small agent to produce and use one is the open problem. Results cover four repositories and one model.

1 Introduction: why look at the map?

In this competition an agent based on Gemma 4 gets a GitHub issue and a repository snapshot, and has to produce a patch that makes hidden tests pass [1]. To help with navigation, the organisers released a *code graph* for every task: each node is a function, method or class, and an edge $A \to B$ means "A calls B". The agent queries it through tools such as `get_code_neighbors`.

Such a *static* call graph is built by reading the code without running it, which is hard in Python: `self.render()` only says that *some* `render` method will run, and frameworks call code through decorators and dispatch tables. My question: **when a failing test runs, which of its calls are invisible in the graph, and does that hurt bug localisation?** I call a runtime call with no matching graph edge a **dark edge**.

This is not a new worry. A public audit ran each repository's whole test suite and found that the graphs hold only 14–21% of runtime call pairs and none of 10,321 resolvable `self.m()` calls [2]; others found no async functions at all [3, 4] and rebuilt the graphs [5, 6]. Outside the competition, DyPyBench found 51% of dynamic Python call edges missing from PyCG's static graphs [7]. This paper is about code-graph generation (the competition's Code Comprehension topic) and releases a reusable tracer. I build on this work and add:

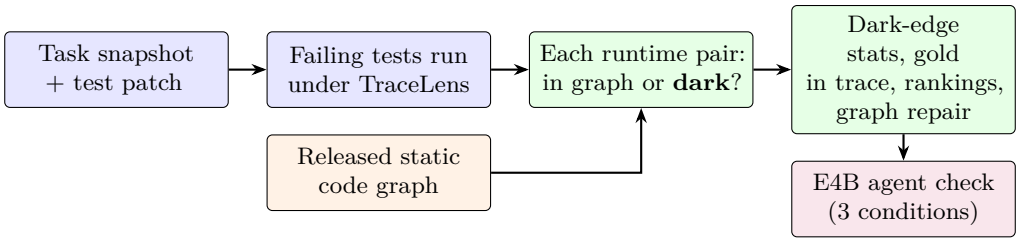


Figure 1: Pipeline. Tracing and comparison need only the standard library; the agent check uses the competition harness.

1. which calls go dark, explaining the repository differences (Section 3);
2. how often the edit location is executed versus reachable in the graph, with an oracle upper bound (Section 4);
3. a static repair that recovers 41% of dark edges but not localisation (Section 5);
4. how much runtime evidence survives without the failing test (Section 6);
5. a pre-registered agent test of a tracing tool for Gemma 4 E4B, which came out null (Section 7).

My hypotheses (H1–H4) were frozen before any analysis, and I report the ones that failed.

2 Setup

Tasks. The competition has 129 tasks from four repositories: fastapi (67), rich (48), requests (13) and httpx (1). For each task I ran the test files touched by the test patch twice: on the base snapshot plus test patch ("pre"), and with the gold fix added ("post"). A task is *valid* if at least one test fails in pre and none fail in post. 103 tasks were valid (fastapi 58, rich 39, requests 5, httpx 1). A looser set of 120 tasks is my sensitivity check; the dark-edge and ranking headlines hold on both. With so few requests and httpx tasks, I make no per-repository claims about them.

Tracer. TraceLens is a small standard-library tracer built on Python's `sys.monitoring` hook. It records every call between repository functions, per test; calls made through library code are linked to the nearest repository caller and marked *lifted*. Overhead was 0.9–1.3×.

Dark edges. I map traced functions to graph node ids (module path plus qualified name); for fastapi 93% of call volume maps; the rest is mostly async methods, which the graph does not contain. A *dark edge* is a runtime pair (caller, callee), both in the graph, with no `calls` edge between them. I count unique pairs per task and average over tasks within a repository.

Gold location. "Gold" functions are those the real fix changes; 89 of the 103 tasks have one in the graph.

Statistics. Confidence intervals (CIs) are 95% percentile intervals from a *task-clustered bootstrap*: whole tasks are resampled 10,000 times. Paired yes/no comparisons use the exact McNemar test, which looks only at the cases where two methods disagree.

Pre-registration. Before the analysis I committed four hypotheses: H1, fastapi has the most dark edges and requests the fewest; H2, trace-based ranking beats static two-hop neighbourhoods by at least 10 points of recall@20; H3, embeddings do not predict dark edges better than names; H4, a tracing tool raises how often a small agent's patch touches a gold file. Everything decided after seeing data is labelled *exploratory*.

	fastapi (58 tasks)	requests (5)	rich (39)
All runtime pairs (headline)	0.47 [0.44, 0.51]	0.52 [0.44, 0.59]	0.81 [0.75, 0.84]
callee is a module-level function	0.14 (66% of pairs)	0.12 (52%)	0.17 (12%)
callee is a method of the same class	1.00 (8%)	1.00 (22%)	1.00 (44%)
callee is a method of another class	1.00 (25%)	1.00 (26%)	0.87 (45%)

Table 1: Share of runtime call pairs that are dark (no graph edge), by type of callee. Brackets: 95% CI. Percentages: share of that repository's pairs.

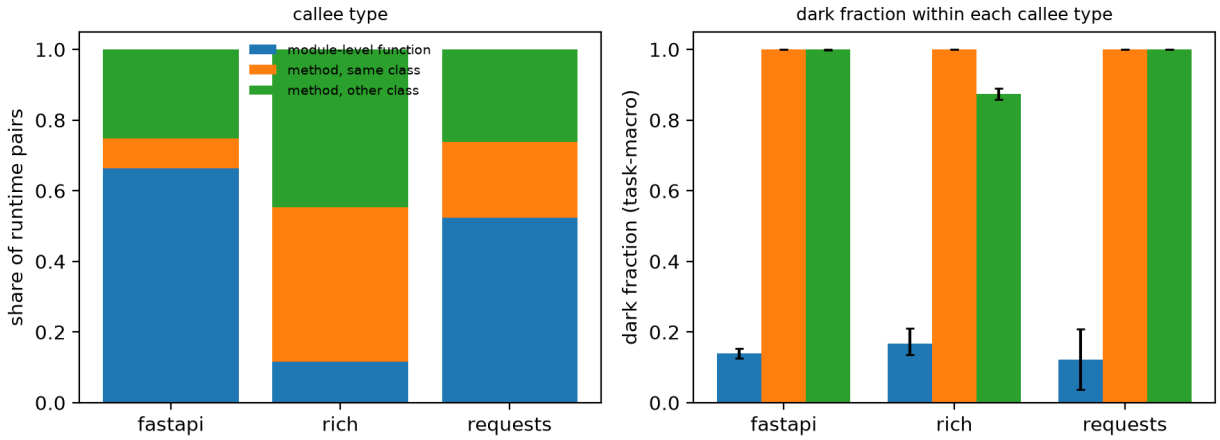


Figure 2: Dark fraction by callee type. Left: what each repository's runtime calls go into. Right: share missing from the graph (task mean, 95% CI).

3 Which calls go dark?

Over all 103 tasks there are 14,059 unique runtime pairs and 10,067 of them (72%) are dark. Per repository, Table 1 shows the pattern that explains almost everything: **a call into a module-level function is usually in the graph, and a call into a method almost never is**. In these graphs a method's calls are mostly attached to its class node instead (68–95% of all `calls` edges touch a class node), and every edge is stored in both directions [2], so direction carries no information.

This also explains the differences between repositories. Rich looks "darker" because 88% of its executed calls go into methods, against 48% for requests and 34% for fastapi. Across tasks, the share of method callees and the dark fraction correlate at $r = 0.94$. This is partly built in, so it supports rather than proves the composition story. **H1 is not supported**: the order is rich > requests > fastapi, and rich exceeds fastapi by 0.33 with a CI that excludes zero.

Reconciling with the public audit (exploratory). Pooled, rich is 84% dark, matching the audit's 79–86%; fastapi is 43% (37% without framework dispatch), likely because I trace only each task's test files and the audit whole suites. A second view confirms the mechanism: if a call counts as covered whenever the graph links the caller to the *callee's class* (two hops via the class node), all three repositories drop to 24–28% dark. The graph knows which class is involved, not which method.

Ranking method (recall@20)	strict, 89 tasks	loose, 104 tasks
Ochiai over failing vs passing tests (oracle)	0.63 [0.54, 0.72]	0.59
Distance from the failure in the runtime call graph (oracle)	0.57 [0.47, 0.67]	0.53
BM25 text search, issue vs function source	0.46 [0.36, 0.56]	0.48
Static graph, two hops from symbols named in the issue	0.26 [0.18, 0.35]	0.27

Table 2: How often a gold function is in the top 20 of each ranking. "Oracle" methods use the tests that define the task, which an agent does not have, so they are upper bounds.

4 Does the missing part matter for localisation?

Is the bug where the tests run? In 83% of the 89 ranked tasks (81% on the looser set), the gold function is executed by the failing tests. In 20% it is reached at run time only through dark edges. But only 24% of tasks have a gold function within three graph hops of the function where the test failed, even with class nodes allowed as stepping stones. The runtime path to the bug usually exists; the graph usually does not show it.

How much could runtime evidence help? I ranked all graph functions per task and measured recall@K: is a gold function in the top K (random tie-breaks, 100 shuffles)? *Ochiai*, a classic spectrum-based fault-localisation score [8, 9], ranks a function high if many failing and few passing tests run it. The others rank by runtime distance from the failure, by BM25 text search (issue vs function source), and by graph distance from symbols named in the issue.

Table 2 shows the result. Ochiai beats the static baseline by 37 points (CI 26 to 49, McNemar $p < 0.001$), so **H2 is supported**. Also, 26 of the 89 issues name no graph symbol, so the static baseline gets nothing there, and Ochiai's lead over BM25 is not stable across task sets.

Embeddings. The released node embeddings separate dark edges from non-edges better than name similarity on held-out repositories (AUC 0.70 vs 0.55–0.58), so **H3 is not supported**, but only modestly, consistent with the near-collapsed embeddings reported on the forum [10].

5 Can the graph be repaired cheaply? (exploratory)

Since most dark edges are method calls, I parsed the source stored in each node and added edges for (R1) `self.m()` and `cls.m()`, resolved through the class and its bases; (R2) `self.m` without a call (properties); (R3) `X(...)` to `X.__init__`.

The repair recovers 4,160 of the 10,067 dark pairs (41%): 74% of same-class method calls, but only 15% of calls into other classes, all through constructors. As a precision check, of the added edges whose two endpoints both ran, 87% were observed at run time. After repair, rich drops from 0.81 to 0.45 dark and fastapi from 0.47 to 0.34 (Figure 3); pooled, the graph then holds 58% of runtime pairs, comparable to the 62–69% of the audit's full `ast` rebuild (different test scope) [2].

Yet the repaired graph **does not improve static localisation**: recall@20 for the two-hop baseline moves from 0.26 to 0.27 (CI of the change -0.01 to $+0.03$). Class nodes already connect sibling methods within two hops, so the new edges add little reachability. The bottleneck is elsewhere: a starting point in the issue text, and a path that only running the code reveals.

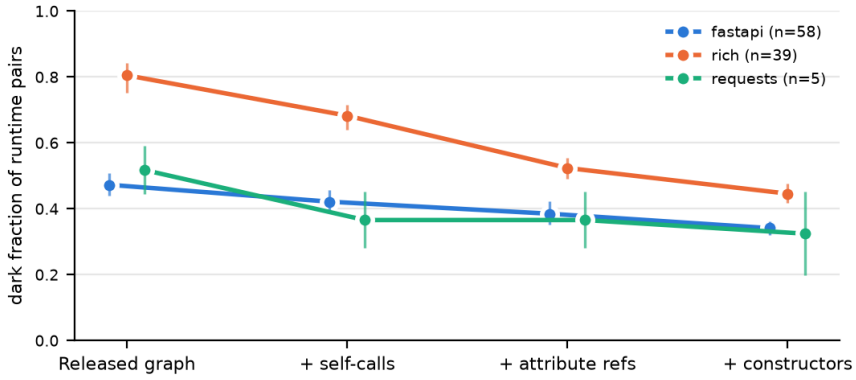


Figure 3: Dark fraction as repair rules are added (task mean, 95% CI). Rules are cumulative.

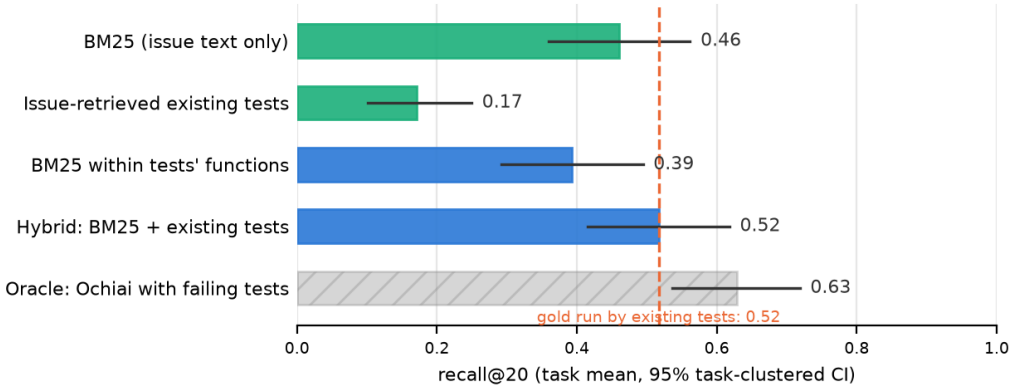


Figure 4: Recall@20 without the failing tests (green, blue) against the oracle (grey). Dashed line: share of tasks where any existing test runs a gold function. 89 tasks, 95% task-clustered CIs.

6 How much runtime evidence survives without the failing test? (exploratory)

An agent does not have the failing tests, so what can it get from tests that *already exist*? I kept only tests that passed before the fix and that the test patch neither adds nor edits (2,652 of 3,384 traced tests; 31 tasks have none) and built a ladder of rankings, with all settings fixed in a dated note before running: BM25 alone; the functions run by the five existing tests most similar to the issue, down-weighting functions that every test runs; BM25 restricted to those functions; and a hybrid that fuses BM25 with the test ranking (reciprocal-rank fusion: each list adds $1/(60 + \text{rank})$).

Figure 4 shows the result. Existing tests run a gold function in only **52%** of tasks, against 83% for the failing tests: the failing test is usually what puts the bug on the runtime path. Test retrieval alone is weak (0.17); the issue matches any test in only 56 tasks. The hybrid reaches 0.52 (equal to the coverage figure only by coincidence), **+0.06 over BM25** (CI −0.02 to +0.14, McNemar $p = 0.27$), about a third of the gap to the oracle with a very wide interval, and it is slightly worse than BM25 at recall@5 (0.28 vs 0.34). The gap to the oracle is a fastapi effect: there BM25 reaches 0.32 and the hybrid 0.38 against an oracle of 0.65, while in rich BM25 already matches the oracle (0.61 vs 0.59). So most of the runtime signal comes from the failing test itself: an agent gains from

	No graph tools	Graph tools	Graph tools + tracer
Gold file touched, all 180 runs	0.72	0.70	0.60
Resolved (hidden tests pass), all 180 runs	0.18	0.18	0.13
Gold file touched, 20 tasks with paired records	0.70	0.70	0.70
Mean tool calls / seconds per run	12.1 / 165	14.0 / 206	14.8 / 204

Table 3: Agent check with Gemma 4 E4B, 30 tasks, 2 seeds. Tracer minus graph tools over all recovered records (task means): -0.07 [-0.27, $+0.10$]; McNemar on the 20 matched seed-0 pairs: 2 vs 2 discordant, $p = 1$.

runtime evidence mainly once it has a failing reproduction.

7 Does a tracing tool help a small agent? (H4)

Design. I ran the competition harness with Gemma 4 E4B on 30 of the 89 ranked tasks (stratified by repository) in three conditions: no graph tools; the three built-in graph tools; graph tools plus TraceLens. Prompts were identical (reproduce, localise, fix, verify, submit) except that the tracer condition asked the agent to trace its reproduction. Two sampling seeds, frozen budget of 10 minutes and 50 tool calls. The pre-registered outcome: does the patch touch a file the gold patch changes?

Result. The tracer did not help: **H4 is not supported** (Table 3). The graph tools did not help either, compared with no graph at all. Uptake was low: the tracer's output reached the agent in only 9 of 32 recovered tracer runs (28%), even though the prompt asked for it right after the reproduction. Runs that did trace hit a gold file about as often as runs that did not (0.67 vs 0.65). Agents used an available debugger in only 0.2–6.2% of runs elsewhere [11], and another entrant saw E4B call a map tool in 1 of 84 runs [12]; on the other side, one team reports more passes with graph tools for E2B on a larger sample [13], and Debug2Fix [14] gains by forcing debugging through a sub-agent. Section 6 suggests why a tracer adds little here: its value depends on a failing reproduction.

Disclosure. A re-executed notebook cell briefly ran two batches at once (up to 8 tasks in parallel instead of 4), and the per-run file was lost with the session. The 180-run summary survived; 92 per-run records were recovered from the printed log. Both are logged in the amendment file.

8 Related work

The competition follows SWE-bench [15]; SWE-agent [16] and Agentless [17] showed that the tool interface and pipeline matter, and LocAgent, RepoGraph and CodexGraph [18–20] give models graph views of code. Spectrum-based fault localisation is decades old [8, 9], and AutoCodeRover [21] already feeds it to an LLM agent. PyCG and Jarvis [22, 23] build better static Python call graphs, and DyPyBench [7] measures what they miss. Within the competition, the audits cited in Section 1 [2, 5, 6, 24, 25] found the coverage gap and that fixes sit far from issue mentions; my additions are the method/function decomposition, the execution-versus-reachability gap, the oracle ceiling and the non-oracle ladder, the repair scored against runtime truth, and the agent test.

9 Limitations

Four repositories, two of which dominate (rich holds 70% of pairs); one tracer configuration; 26 tasks without a valid before/after run; nested and async functions never map (the hosts state that no private task touches async code [4]). The agent check has 30 tasks, one model, one prompt and partially recovered records. Holm correction across H1–H4 changes nothing: H2 ($p < 0.001$) survives.

10 Conclusion and resources

Takeaways for agent builders. (1) Do not ask the released graph about method calls; grep for them or index `self.m()` calls. (2) A denser graph does not localise better; start from text search over the issue. (3) Runtime evidence pays off through a failing reproduction; adding existing tests to BM25 gains only about 6 points. (4) Small agents rarely call optional tools, so putting the evidence in the prompt is the natural next test (not tried here). (5) For graph builders: resolve `self.m()`, attribute and constructor calls; this alone recovers 41% of dark edges.

Code and data. TraceLens, the agent skill, all analysis scripts, the frozen protocol (`configs/protocol.yaml`, tag `protocol-v1`), the amendment log, and aggregate result tables are at [26]. A public Kaggle notebook (Project Links) installs TraceLens, shows a dark edge on a toy package, runs the unit tests and redraws Figures 3–4 from the released aggregates; the PDF is attached to the release. Competition data and traces are not redistributed. Software: Python 3.12, seed 0.

AI assistance. I used an AI assistant to help write analysis code and to help draft and edit this text. The study design, experiments, numbers and conclusions are mine and were checked against the logged outputs.

References

[1] Kaggle. Google – The Gemma 4 Developer Agent: Paper track. https://www.kaggle.com/competitions/gemma-4-developer-agent-paper, 2026.

[2] akhilchinta1505. Code graph audit notebook and call-trace dataset (notes on the graph tools from auditing all 127 graphs). https://www.kaggle.com/code/akhilchinta1505/code-graph-audit, 2026.

[3] aghasalim. gemma4-code-graph-localization. https://github.com/aghasalim/gemma4-code-graph-localization, 2026.

[4] Competition hosts. Note on code graphs for async functions (host post): private tasks touch no async def. Gemma 4 Developer Agent forum, https://www.kaggle.com/competitions/gemma-4-developer-agent/discussion/745998, 2026.

[5] saurin16. Mind the map: Auditing and repairing the released Gemma 4 code graphs. https://github.com/saurin16/mind-the-map, 2026.

[6] danielmantiglia. What should a code graph contain? (codegraph-loc). https://github.com/danielmantiglia/codegraph-loc, 2026.

[7] Islem Bouzenia, Bajaj Piyush Krishan, and Michael Pradel. DyPyBench: A benchmark of executable Python software. *Proceedings of the ACM on Software Engineering*, 1(FSE), 2024. doi: 10.1145/3643742.

[8] Rui Abreu, Peter Zoeteweij, and Arjan J. C. van Gemund. On the accuracy of spectrum-based fault localization. In *Testing: Academic and Industrial Conference Practice and Research Techniques (TAIC PART-MUTATION)*, pages 89–98, 2007. doi: 10.1109/TAICPART.2007.4344104.

[9] W. Eric Wong, Ruizhi Gao, Yihao Li, Rui Abreu, and Franz Wotawa. A survey on software fault localization. *IEEE Transactions on Software Engineering*, 42(8):707–740, 2016. doi: 10.1109/TSE.2016.2521368.

[10] Kaggle competitors. Embeddings for search_similar_code appear near-collapsed. Gemma 4 Developer Agent forum, https://www.kaggle.com/competitions/gemma-4-developer-agent/discussion/744040, 2026.

[11] Christoph Daxerer, Johann Glock, Christian Macho, Rudolf Ramler, Martin Pinzger, and Josef Pichler. Do coding agents still need a debugger? Frontier-model-based fault localization at the accuracy ceiling. *arXiv preprint arXiv:2610.36015*, 2026.

[12] clavis-systems. Who reads the code map? push vs pull graph context for gemma 4 e4b. https://github.com/clavis-systems/who-reads-the-code-map, 2026.

[13] nakranivaibhav. gemma-4-developer-agent: graph-tool a/b with gemma 4 e2b. https://github.com/nakranivaibhav/gemma-4-developer-agent, 2026.

[14] Spandan Garg and Yufan Huang. Debug2Fix: Supercharging coding agents with interactive debugging capabilities. *arXiv preprint arXiv:2602.18571*, 2026.

[15] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.06770.

[16] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37, 2024. arXiv:2405.15793.

[17] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Demystifying LLM-based software engineering agents. *Proceedings of the ACM on Software Engineering*, 2(FSE), 2025. doi: 10.1145/3715754. arXiv:2407.01489 ("Agentless").

[18] Zhaoling Chen, Xiangru Tang, Gangda Deng, Fang Wu, Jialong Wu, Zhiwei Jiang, Viktor Prasanna, Arman Cohan, and Xingyao Wang. LocAgent: Graph-guided LLM agents for code localization. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025. arXiv:2503.09089.

[19] Siru Ouyang, Wenhao Yu, Kaixin Ma, Zilin Xiao, Zhihan Zhang, Mengzhao Jia, Jiawei Han, Hongming Zhang, and Dong Yu. RepoGraph: Enhancing AI software engineering with repository-level code graph. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2410.14684.

[20] Xiangyan Liu, Bo Lan, Zhiyuan Hu, Yang Liu, Zhicheng Zhang, Fei Wang, Michael Shieh, and Wenmeng Zhou. CodexGraph: Bridging large language models and code repositories via code graph databases. In *Proceedings of NAACL (Long Papers)*, pages 142–160, 2025. doi: 10.18653/v1/2025.naacl-long.7.

[21] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. AutoCodeRover: Autonomous program improvement. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, pages 1592–1604, 2024. doi: 10.1145/3650212.3680384.

[22] Vitalis Salis, Thodoris Sotiropoulos, Panos Louridas, Diomidis Spinellis, and Dimitris Mitropoulos. PyCG: Practical call graph generation in Python. In *IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*, pages 1646–1657, 2021. doi: 10.1109/ICSE43902.2021.00146.

[23] Kaifeng Huang, Yixuan Yan, Bihuan Chen, Zixin Tao, and Xin Peng. Scalable and precise application-centered call graph construction for Python. *arXiv preprint arXiv:2305.05949*, 2024.

[24] earthmaker. When count: 0 means "not indexed". https://github.com/earthmaker/count-zero-not-indexed, 2026.

[25] satiricalguru. Gemma-graph-nav: hop distance from issue anchors to the fix. https://github.com/satiricalguru/Gemma-graph-nav, 2026.

[26] Soumi Sharma. darkedges: TraceLens tracer, analysis pipeline and aggregate results. https://github.com/soum-stacks/darkedges-paper, tag paper-v1, 2026.